



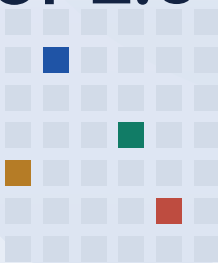
ISCA 2026 · TUTORIAL

Extending Ramulator 2.0

Toward higher-fidelity, secure, and
energy-aware DRAM simulation

Prashant J. Nair

The University of British Columbia · STAR Lab



Ramulator 2.0 today

A fast, cycle-accurate, open-source DRAM simulator — modular, extensible, broadly standard-compliant; driven by CPU traces or a front-end simulator.



STANDARDS MODELED

DDR3 / 4 / 5

LPDDR4 / 5

GDDR6

HBM2 / 3

THE STARTING POINT

Ramulator 2.0 today

A fast, cycle-accurate, open-source DRAM simulator — modular, extensible, broadly standard-compliant; driven by CPU traces or a front-end simulator.



STANDARDS MODELED

DDR3 / 4 / 5

LPDDR4 / 5

GDDR6

HBM2 / 3



Modular core

Interfaces vs. implementations



Fast & validated

Cycle-accurate, competitive speed



Easy to extend

Add standards / policies as plugins

Why extend Ramulator now?

Ramulator 2.0 is fast and modular. Yet we would like several additional effects that dominate modern DRAM behaviour to be modelled. Enabling this unlocks new research.

Why extend Ramulator now?

Ramulator 2.0 is fast and modular. Yet we would like several additional effects that dominate modern DRAM behaviour to be modelled. Enabling this unlocks new research.



Higher accuracy

Mapping, page policy, refresh and scheduling shape latency and bandwidth.



Security & reliability

RowHammer and ECC are first-class concerns, yet cannot be evaluated end-to-end.



Shared platform

One faithful, configurable simulator lets the community compare ideas fairly.

Why extend Ramulator now?

Ramulator 2.0 is fast and modular. Yet we would like several additional effects that dominate modern DRAM behaviour to be modelled. Enabling this unlocks new research.



Higher accuracy

Mapping, page policy, refresh and scheduling shape latency and bandwidth.



Security & reliability

RowHammer and ECC are first-class concerns, yet cannot be evaluated end-to-end.



Shared platform

One faithful, configurable simulator lets the community compare ideas fairly.

Goal — make Ramulator a **higher-fidelity**, **security-aware**, and **energy-aware** platform that the whole community can build on.

Four areas to improve

Each extension turns a coarse or missing model and makes it into a configurable one.

Four areas to improve

Each extension turns a coarse or missing model and makes it into a configurable one.



AREA 1

Modeling fidelity

Mapping · page policy · refresh · scheduling



AREA 2

Front-end integration

gem5 · ChampSim · maintained wrappers



AREA 3

Security & reliability

RowHammer bit-flips · ECC & errors



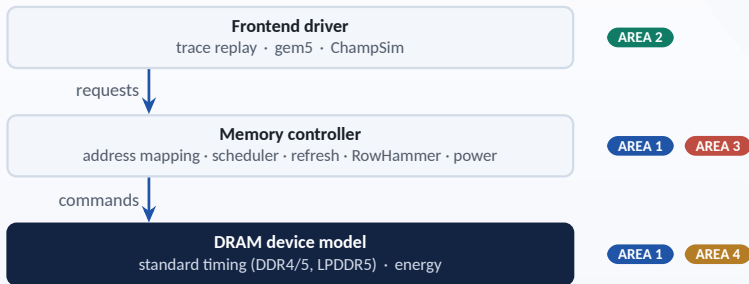
AREA 4

Power & energy

Per-command energy · power model

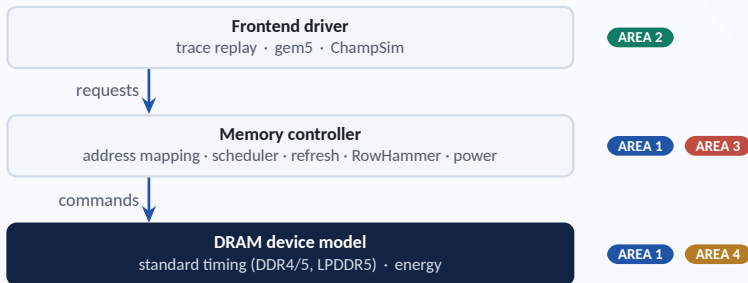
Inside Ramulator 2.0

Ramulator 2.0 is modular — each extension in this talk is a plugin at a defined layer.



Inside Ramulator 2.0

Ramulator 2.0 is modular — each extension in this talk is a plugin at a defined layer.



One codebase — Area 1 focuses on the controller & device, Area 2 highlights the the frontend, Area 3 adds controller plugins, and Area 4 enriches the device energy model.

Modeling Fidelity & Policy Completeness

Make the controller and device model what Industry Products do.

Address mapping

Row-buffer policy

Refresh & RFM

Scheduling



1 • Modeling fidelity

2 • Front-end

3 • Security

4 • Power & energy

DRAM organization & timing

Every timing model starts from the same hierarchy and the same handful of command constraints.



These are the parameters Ramulator's DRAM *standard* classes encode. The aim is to get them to model industry products precisely.

DRAM organization & timing

Every timing model starts from the same hierarchy and the same handful of command constraints.



ACCESS LATENCIES

t_{RCD}: ACT → column **CL**: RD → data

t_{RP}: PRE → ACT **t_{RAS}**: ACT → PRE

t_{RC}: ACT → ACT, same bank (= t_{RAS}+t_{RP})

SPACING CONSTRAINTS

t_{RRD} / **t_{FAW}**: activation rate, 4-ACT window

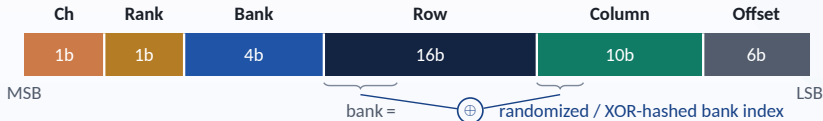
t_{CCD_S/L}: column-to-column (bank group)

t_{WR}: write recovery **t_{WTR}**: write → read

These are the parameters Ramulator's DRAM *standard* classes encode. The aim is to get them to model industry products precisely.

Address mapping

Address bits decide how accesses spread across banks and effective workload parallelism.



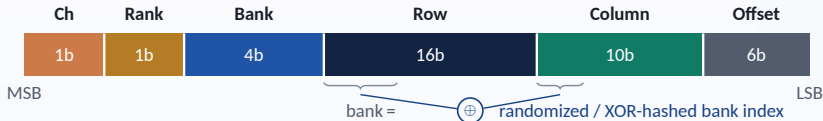
DRAMA · *USENIX Sec.* 2016

ZenHammer · *USENIX Sec.* 2024

Rubix · *ASPLOS* 2024

Address mapping

Address bits decide how accesses spread across banks and effective workload parallelism.



CURRENTLY IN RAMULATOR

A few fixed, linear mappings — not the reverse-engineered functions real Intel / AMD CPUs use.

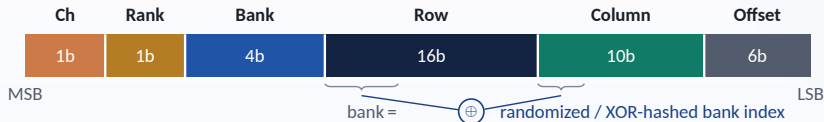
DRAMA · *USENIX Sec.* 2016

ZenHammer · *USENIX Sec.* 2024

Rubix · *ASPLOS* 2024

Address mapping

Address bits decide how accesses spread across banks and effective workload parallelism.



CURRENTLY IN RAMULATOR

A few fixed, linear mappings — not the reverse-engineered functions real Intel / AMD CPUs use.

WHAT TO ADD

Vendor maps (Intel, AMD Zen) plus **XOR-hashed & randomized** bank hashing.

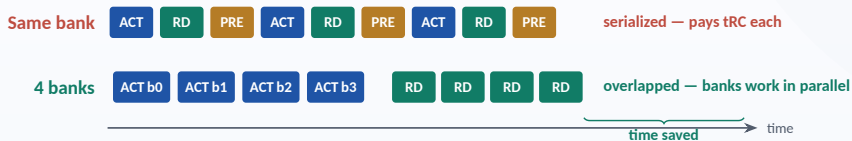
DRAMA · USENIX Sec. 2016

ZenHammer · USENIX Sec. 2024

Rubix · ASPLOS 2024

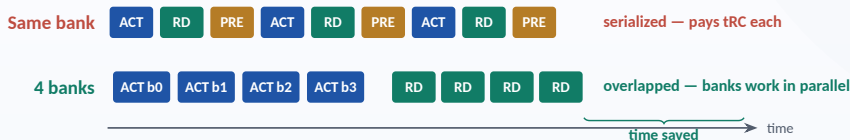
Why mapping matters: bank parallelism

Spreading consecutive accesses across banks overlaps their latencies; piling them on one bank serializes them. The address map decides which you get.



Why mapping matters: bank parallelism

Spreading consecutive accesses across banks overlaps their latencies; piling them on one bank serializes them. The address map decides which you get.

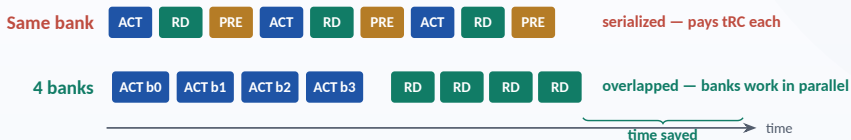


BANK CONFLICTS

A poor map sends a hot stream to one bank — every access waits on t_{RC} , so latency stacks up.

Why mapping matters: bank parallelism

Spreading consecutive accesses across banks overlaps their latencies; piling them on one bank serializes them. The address map decides which you get.



BANK CONFLICTS

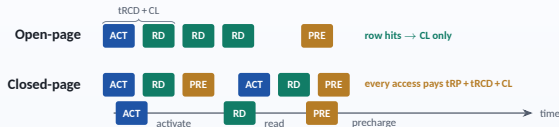
A poor map sends a hot stream to one bank — every access waits on **tRC**, so latency stacks up.

BANK-LEVEL PARALLELISM

A good map scatters accesses across banks and channels, so the device serves them **concurrently**.

Row-buffer (page) policy

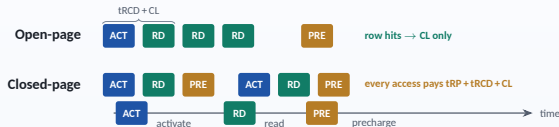
Whether the controller keeps a row open after an access changes every subsequent latency — a row hit pays only CL , a row miss pays $t_{RP} + t_{RCD} + CL$.



FR-FCFS / Memory Access Scheduling — Rixner, Dally, Kapasi, Mattson, Owens, ISCA 2000

Row-buffer (page) policy

Whether the controller keeps a row open after an access changes every subsequent latency — a row hit pays only CL , a row miss pays $t_{RP} + t_{RCD} + CL$.



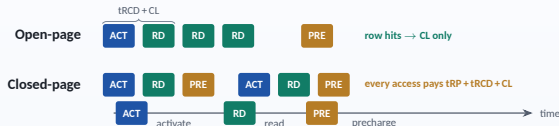
CURRENTLY IN RAMULATOR

Open-page, closed-page and FR-FCFS with a fixed row-hit cap — the policy is chosen statically.

FR-FCFS / Memory Access Scheduling — Rixner, Dally, Kapasi, Mattson, Owens, ISCA 2000

Row-buffer (page) policy

Whether the controller keeps a row open after an access changes every subsequent latency — a row hit pays only CL , a row miss pays $t_{RP} + t_{RCD} + CL$.



CURRENTLY IN RAMULATOR

Open-page, closed-page and FR-FCFS with a fixed row-hit cap — the policy is chosen statically.

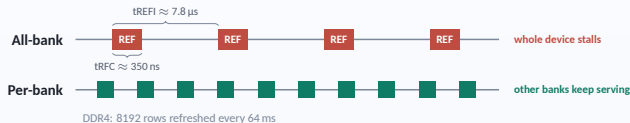
WHAT TO ADD

Adaptive policies that switch open ↔ closed by access pattern, as modern servers do.

FR-FCFS / Memory Access Scheduling — Rixner, Dally, Kapasi, Mattson, Owens, ISCA 2000

Refresh & refresh management

Refresh reduces bandwidth. The granularity of refresh decides how much bandwidth is lost. JEDEC RFM adds a second function to effectively throttle activations to bound RowHammer.

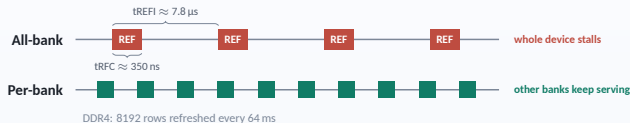


RAIDR — Liu, Jaiyen, Veras, Mutlu, ISCA 2012

RFM — JEDEC DDR5 (JESD79-5)

Refresh & refresh management

Refresh reduces bandwidth. The granularity of refresh decides how much bandwidth is lost. JEDEC RFM adds a second function to effectively throttle activations to bound RowHammer.



CURRENTLY IN RAMULATOR

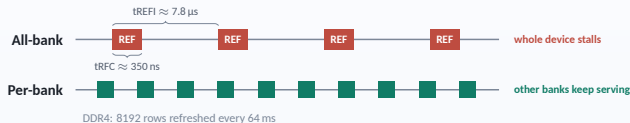
All-bank **REFab** only — the device pauses each **tREFI**. No RFM.

RAIDR — Liu, Jaiyen, Veras, Mutlu, ISCA 2012

RFM — JEDEC DDR5 (JESD79-5)

Refresh & refresh management

Refresh reduces bandwidth. The granularity of refresh decides how much bandwidth is lost. JEDEC **RFM** adds a second function to effectively throttle activations to bound RowHammer.



CURRENTLY IN RAMULATOR

All-bank **REFab** only — the device pauses each **tREFI**. No **RFM**.

WHAT TO ADD

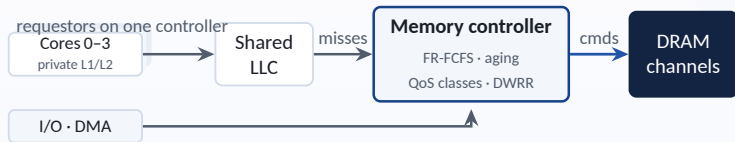
Per-bank & fine-grained (**1x/2x/4x**) refresh, postponement, JEDEC **RFM**.

RAIDR — Liu, Jaiyen, Veras, Mutlu, *ISCA* 2012

RFM — JEDEC DDR5 (JESD79-5)

Scheduling policy

Many on-chip requestors may flow through the same controller.

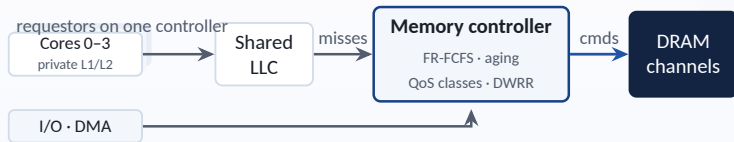


TCM — Kim, Papamichael, Mutlu, Harchol-Balter, *MICRO* 2010

QoS — ARM AMBA AXI

Scheduling policy

Many on-chip requestors may flow through the same controller.



CURRENTLY IN RAMULATOR

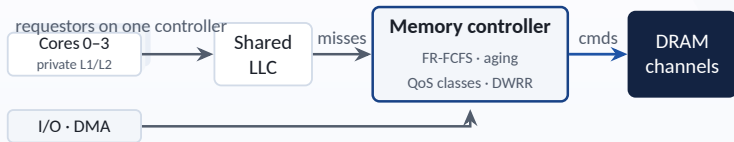
The FR-FCFS family — row hits first, then oldest. No QoS classes.

TCM — Kim, Papamichael, Mutlu, Harchol-Balter, *MICRO* 2010

QoS — ARM AMBA AXI

Scheduling policy

Many on-chip requestors may flow through the same controller.



CURRENTLY IN RAMULATOR

The FR-FCFS family — row hits first, then oldest. No QoS classes.

WHAT TO ADD

QoS priority, **aging** to stop starvation, and partitioning with Deficit Weighted Round Robin (**DWRR**).

TCM — Kim, Papamichael, Mutlu, Harchol-Balter, *MICRO* 2010

QoS — ARM AMBA AXI

Front-End Simulator Integration

Drive Ramulator from a other core models and make the interface seamless.

gem5

ChampSim

Maintained wrappers

Matched stats

1 • Modeling fidelity

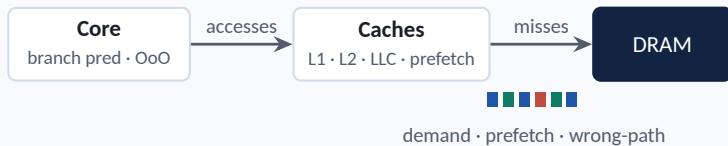
2 • Front-end

3 • Security

4 • Power & energy

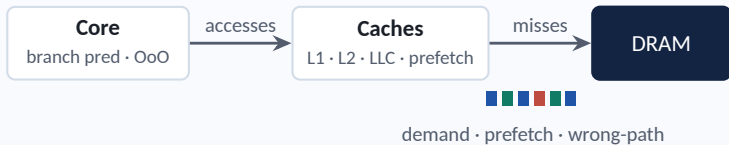
Why front-end fidelity matters

What reaches DRAM is whatever the core and caches let through — a fixed trace hides the wrong-path, prefetch, and filtering effects of a real front-end.



Why front-end fidelity matters

What reaches DRAM is whatever the core and caches let through — a fixed trace hides the wrong-path, prefetch, and filtering effects of a real front-end.



Branch prediction

Wrong-path loads from mispredictions still reach DRAM.



Prefetching

Prefetchers reshape the demand / prefetch mix.

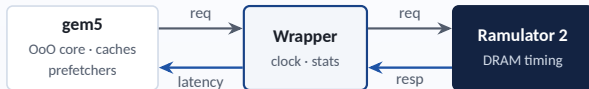


Cache hierarchy

L1 / L2 / LLC decide which accesses reach DRAM.

Coupling with gem5

gem5 brings an *alternate* model for out-of-order core, caches, and prefetchers. However, wiring it to Ramulator today is manual and drifts out of date.

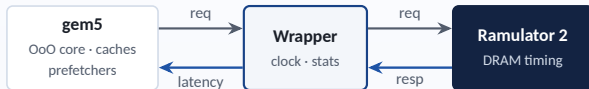


gem5 — Binkert et al., *SIGARCH Comput. Archit. News* 2011

gem5.org

Coupling with gem5

gem5 brings an *alternate* model for out-of-order core, caches, and prefetchers. However, wiring it to Ramulator today is manual and drifts out of date.



CURRENTLY IN RAMULATOR

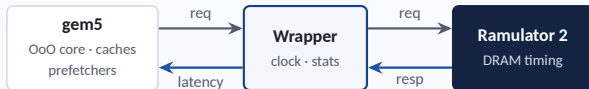
Coupling lives in a README — manual patching that drifts out of date with each release.

gem5 — Binkert et al., *SIGARCH Comput. Archit. News* 2011

gem5.org

Coupling with gem5

gem5 brings an *alternate* model for out-of-order core, caches, and prefetchers. However, wiring it to Ramulator today is manual and drifts out of date.



CURRENTLY IN RAMULATOR

Coupling lives in a README — manual patching that drifts out of date with each release.

WHAT TO ADD

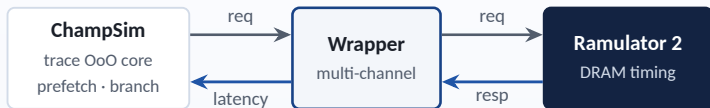
A **maintained wrapper** with matched timing and **merged statistics** across both tools.

gem5 — Binkert et al., *SIGARCH Comput. Archit. News* 2011

gem5.org

Coupling with ChampSim

ChampSim has a trace-driven core for prefetcher and branch-prediction research. Some schools use this tool for teaching (say UBC). However, Ramulator-2.0 ships no built-in bridge to it.

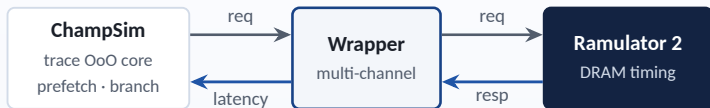


ChampSim · github.com/ChampSim/ChampSim

TPRAC · ChampSim × Ramulator 2 (STAR-Lab)

Coupling with ChampSim

ChampSim has a trace-driven core for prefetcher and branch-prediction research. Some schools use this tool for teaching (say UBC). However, Ramulator-2.0 ships no built-in bridge to it.



CURRENTLY IN RAMULATOR

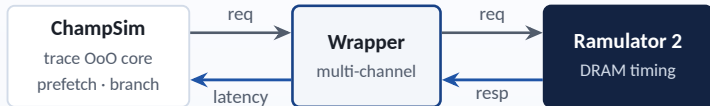
No built-in coupling in Ramulator 2 — every group wires up its own ad-hoc bridge.

ChampSim · github.com/ChampSim/ChampSim

TPRAC · ChampSim × Ramulator 2 (STAR-Lab)

Coupling with ChampSim

ChampSim has a trace-driven core for prefetcher and branch-prediction research. Some schools use this tool for teaching (say UBC). However, Ramulator-2.0 ships no built-in bridge to it.



CURRENTLY IN RAMULATOR

No built-in coupling in Ramulator 2 — every group wires up its own ad-hoc bridge.

WHAT TO ADD

A **maintained wrapper**, verified **multi-channel**, with matched interfaces and stats.

ChampSim · github.com/ChampSim/ChampSim

TPRAC · ChampSim × Ramulator 2 (STAR-Lab)

Security & Reliability

Move from *costing* mitigations to *testing* whether they actually work.

RowHammer bit-flips

ECC & error modeling

What it enables

A decorative graphic consisting of a grid of small squares, some of which are slightly darker than others, creating a subtle pattern.

1 • Modeling fidelity

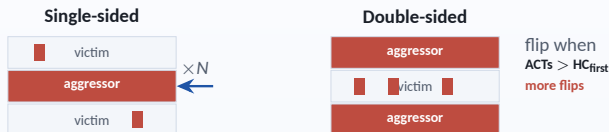
2 • Front-end

3 • Security

4 • Power & energy

RowHammer

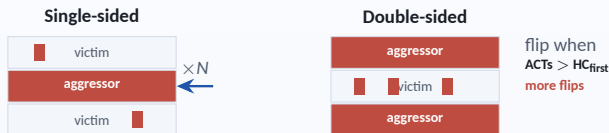
Hammering an **aggressor** row leaks charge from its **neighbours**. However, past a threshold, bit flips occur in the victim rows. Today Ramulator can model the cost of a mitigation. But it cannot test it.



RowHammer / PARA · ISCA 2014 Graphene · MICRO 2020 Hydra · ISCA 2022

RowHammer

Hammering an **aggressor** row leaks charge from its **neighbours**. However, past a threshold, bit flips occur in the victim rows. Today Ramulator can model the cost of a mitigation. But it cannot test it.



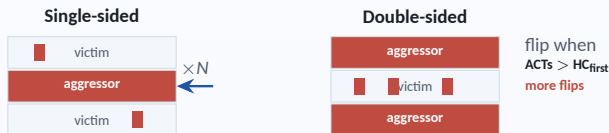
CURRENTLY IN RAMULATOR

Mitigations like PARA, TRR and Hydra run as plugins but there is no mechanism to test their security.

RowHammer / PARA · ISCA 2014 Graphene · MICRO 2020 Hydra · ISCA 2022

RowHammer

Hammering an **aggressor** row leaks charge from its **neighbours**. However, past a threshold, bit flips occur in the victim rows. Today Ramulator can model the cost of a mitigation. But it cannot test it.



CURRENTLY IN RAMULATOR

Mitigations like PARA, TRR and Hydra run as plugins but there is no mechanism to test their security.

WHAT TO ADD

A **bit-flip model** + realistic hammer patterns to test whether a mitigation truly stops flips.

RowHammer / PARA · ISCA 2014 Graphene · MICRO 2020 Hydra · ISCA 2022

RowHammer: mitigations to test

Thresholds keep falling — HC_{first} dropped from ~ 139 K (DDR3) toward a few thousand — so a faithful model must check each defense class actually works.

RowHammer: mitigations to test

Thresholds keep falling — HC_{first} dropped from ~ 139 K (DDR3) toward a few thousand — so a faithful model must check each defense class actually works.



Probabilistic

PARA — on each activation, refresh a neighbour with probability p . Cheap, stateless.



In-DRAM TRR

On-die vendor logic — opaque, and already broken (*TRRespass*, *Blacksmith*).



Counter-based

Graphene · **Hydra** · **PRAC** count activations, act at a threshold. Accurate, costlier.

RowHammer: mitigations to test

Thresholds keep falling — HC_{first} dropped from ~ 139 K (DDR3) toward a few thousand — so a faithful model must check each defense class actually works.



Probabilistic

PARA — on each activation, refresh a neighbour with probability p . Cheap, stateless.



In-DRAM TRR

On-die vendor logic — opaque, and already broken (*TRRespass*, *Blacksmith*).



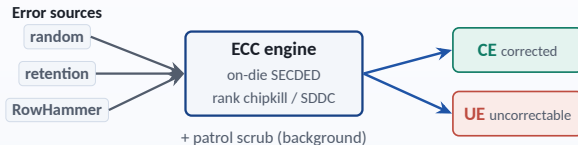
Counter-based

Graphene · Hydra · PRAC count activations, act at a threshold. Accurate, costlier.

The simulator's job — does each defense **stop the flips** at a realistic threshold, and what **performance**, **energy** and **area** does it cost?

ECC & error modeling

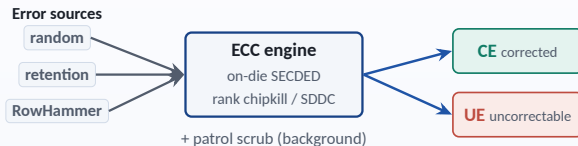
DRAM bits fail and ECC helps mitigate these. There is a latency and bandwidth cost.



DRAM Errors in the Wild — Schroeder, Pinheiro, Weber, *SIGMETRICS* 2009 · On-die ECC — JEDEC DDR5

ECC & error modeling

DRAM bits fail and ECC helps mitigate these. There is a latency and bandwidth cost.



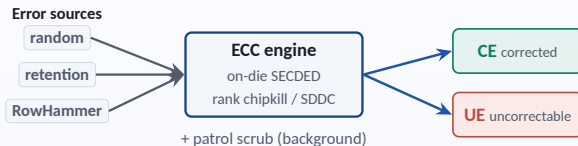
CURRENTLY IN RAMULATOR

No error or reliability model — every access is assumed perfect, ECC is free.

DRAM Errors in the Wild — Schroeder, Pinheiro, Weber, *SIGMETRICS* 2009 · On-die ECC — JEDEC DDR5

ECC & error modeling

DRAM bits fail and ECC helps mitigate these. There is a latency and bandwidth cost.



CURRENTLY IN RAMULATOR

No error or reliability model — every access is assumed perfect, ECC is free.

WHAT TO ADD

Inject errors, model ECC latency + scrub cost, track **CE vs UE** rates.

DRAM Errors in the Wild — Schroeder, Pinheiro, Weber, *SIGMETRICS* 2009 · On-die ECC — JEDEC DDR5

Power & Energy Analysis

Turn a coarse power estimate into per-command, per-state energy.

Per-command energy

Power state machine

DRAMPower

IDD params

1 • Modeling fidelity

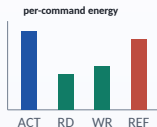
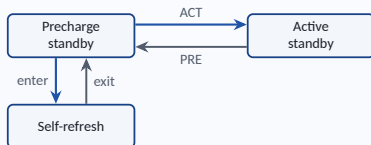
2 • Front-end

3 • Security

4 • Power & energy

Detailed power modeling

Energy depends on each command and the rank's idle power state.

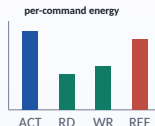
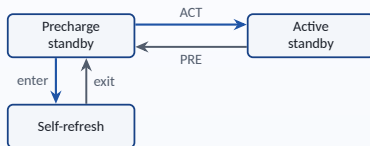


DRAMPower · github.com/tukl-msd/DRAMPower

Micron · DDR System Power Calculator

Detailed power modeling

Energy depends on each command and the rank's idle power state.



CURRENTLY IN RAMULATOR

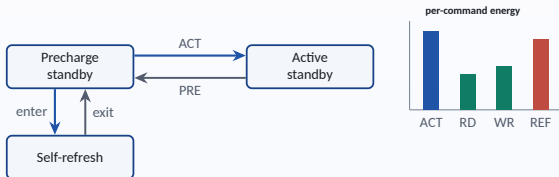
A single coarse power estimate. There is no per-command or per-state energy breakdown.

DRAMPower · github.com/tukl-msd/DRAMPower

Micron · DDR System Power Calculator

Detailed power modeling

Energy depends on each command and the rank's idle power state.



CURRENTLY IN RAMULATOR

A single coarse power estimate. There is no per-command or per-state energy breakdown.

WHAT TO ADD

Per-command + per-state energy from a detailed model (e.g. **DRAMPower**).

DRAMPower · github.com/tukl-msd/DRAMPower

Micron · DDR System Power Calculator

WRAP-UP

Four extensions, one platform

Each enables additional configurability.

Four extensions, one platform

Each enables additional configurability.



AREA 1 • MODELING FIDELITY

Mapping • pages • refresh • scheduling

→ results that track real DRAM



AREA 2 • FRONT-END INTEGRATION

gem5 & ChampSim • matched stats

→ real access streams, maintained



AREA 3 • SECURITY & RELIABILITY

Bit-flip model • ECC & error injection

→ test mitigations, not just cost them



AREA 4 • POWER & ENERGY

Per-command & per-state energy

→ energy you can trust

Selected references

AREA 1 • MODELING FIDELITY

Pessl, Gruss, Maurice, Schwarz, Mangard. *DRAMA*. USENIX Security 2016.

Jattke et al. *ZenHammer*. USENIX Security 2024.

Saxena, Mathur, Qureshi. *Rubix*. ASPLOS 2024.

Rixner, Dally, Kapasi, Mattson, Owens.

Memory Access Scheduling. ISCA 2000.

Liu, Jaiyen, Veras, Mutlu. *RAIDR*. ISCA 2012.

Kim, Papamichael, Mutlu, Harchol-Balter.

Thread Cluster Memory Scheduling. MICRO 2010.

AREA 2 • FRONT-END INTEGRATION

Binkert et al. *The gem5 Simulator*. SIGARCH CAN, 2011.

ChampSim. github.com/ChampSim/ChampSim

TPRAC — ChampSim × Ramulator 2 (STAR Lab).

AREA 3 • SECURITY & RELIABILITY

Kim et al. *Flipping Bits in Memory Without Accessing Them* (RowHammer, PARA). ISCA 2014.

Park et al. *Graphene*. MICRO 2020.

Qureshi, Rohan, Saileshwar, Nair. *Hydra*. ISCA 2022.

Schroeder, Pinheiro, Weber. *DRAM Errors in the Wild*.

SIGMETRICS 2009.

AREA 4 • POWER & ENERGY

DRAMPower. github.com/tukl-msd/DRAMPower

Micron. *DDR System Power Calculator*.

PLATFORM

Luo et al. *Ramulator 2.0*. IEEE CAL 2024.

github.com/CMU-SAFARI/ramulator2

Thank you

Questions & discussion welcome.

Prashant J. Nair The University of British Columbia · STAR Lab

